⚔️ Guilda de Aventureiros — Guia Didático

Entendendo, parte por parte, como o projeto funciona.

---

1. A ideia geral

O projeto é uma **API REST**: um programa que fica rodando num servidor, guarda dados num **banco de dados** e responde a pedidos (requisições) feitos por outros programas — o navegador, o Thunder Client, o Postman etc.

Pense numa taverna:

Na taverna	No projeto
Cliente que faz o pedido	Navegador / Thunder Client
Balcão de atendimento	**Rotas** (`routes/`)
Atendente que prepara o pedido	**Controllers** (`controllers/`)
Cofre onde tudo é guardado	**Banco SQLite** (`guilda.db`)

O caminho de uma requisição

```
GET /herois
   ↓
server.js            sobe o servidor na porta 3000
   ↓
app.js               passa por middlewares (cors, express.json)
   ↓
routes/herois.routes   "GET na raiz de /herois? chame controller.listar"
   ↓
controllers/herois     executa o SELECT no banco e responde em JSON
   ↓
config/database        a conexão com o SQLite
```

Por que separar em camadas? Cada arquivo tem uma responsabilidade. Se o banco mudar, mexe-se em um lugar. Se uma regra mudar, mexe-se no controller. O código fica fácil de achar e de manter.

---

2. Estrutura de pastas

```
guilda-aventureiros-api/
├── backend/
│   ├── server.js              → liga o servidor
│   ├── package.json           → dependências e scripts
│   ├── database/guilda.db     → arquivo do banco (criado sozinho)
│   └── src/
│       ├── app.js             → configura o Express
│       ├── config/database.js → conexão + criação das tabelas
│       ├── routes/            → 4 arquivos de rotas
│       └── controllers/       → 4 arquivos com a lógica
```

```
└── frontend/
    ├── index.html
    └── src/css, src/js        → interface (bônus)
```

---

3. O banco de dados (`config/database.js`)

Conceitos

- **Tabela:** como uma planilha. Cada linha é um registro.
- **Chave primária (PRIMARY KEY):** o identificador único de cada linha (`id`). Com `AUTOINCREMENT` , o banco gera o número sozinho.
- **Chave estrangeira (FOREIGN KEY):** uma coluna que aponta para a chave primária de *outra* tabela. É assim que um item sabe a qual herói pertence.

As três tabelas

```
herois                itens                  missoes
______                _____                  _______
id  (PK)              id  (PK)               id  (PK)
nome                  nome                   nome
classe                tipo                   status
nivel                 poder                  heroi_id (FK → herois.id)
                      heroi_id (FK → herois.id)
```

Relação: **um herói tem vários itens e várias missões** (relação 1-para-N).

Detalhes importantes do código

- `db.pragma("foreign_keys = ON")` — o SQLite **ignora** chaves estrangeiras por padrão. Sem esta linha, o banco aceitaria um item de um herói que não existe.
- `ON DELETE CASCADE` — ao apagar um herói, seus itens e missões são apagados junto, evitando registros "órfãos".
- `CHECK (status IN ('em andamento','concluída'))` — o próprio banco impede qualquer outro valor de status.
- `CREATE TABLE IF NOT EXISTS` — pode rodar o programa quantas vezes quiser; as tabelas só são criadas se ainda não existirem.
- `fs.mkdirSync(..., { recursive: true })` — o SQLite cria o arquivo `.db` , mas **não** a pasta. Por isso criamos a pasta antes.
- O `better-sqlite3` é **síncrono**: você chama e já recebe o resultado, sem `async/await` .

Os três métodos que você mais usa

Método	Para quê	Retorna
`.all()`	SELECT de vários registros	array
`.get()`	SELECT de um registro	objeto ou `undefined`
`.run()`	INSERT, UPDATE, DELETE	`{ changes, lastInsertRowid }`

Segurança: o `?`

```
db.prepare("SELECT * FROM herois WHERE id = ?").get(req.params.id);
```

O `?` é um **parâmetro**. O valor é enviado separado do texto SQL, o que impede **SQL Injection** (alguém digitar código SQL malicioso num campo). **Nunca** monte SQL concatenando strings com dados do usuário.

---

4. O servidor (`server.js` e `app.js`)

`server.js`

Só faz uma coisa: importa o `app` e o coloca para escutar na porta 3000 (`app.listen`).

`app.js`

Monta a aplicação Express, na ordem:

1. `require("./config/database")` — garante que o banco e as tabelas existam ao iniciar.
2. `app.use(cors())` — **middleware** que libera acesso de outras origens (útil se o front rodar em outra porta).
3. `app.use(express.json())` — middleware que lê o corpo JSON da requisição e o coloca em `req.body` . **Sem ele, `req.body` seria `undefined` .**
4. `app.use(express.static(...))` — serve a pasta `frontend` , então `http://localhost:3000/` abre a interface.
5. `app.use("/herois", heroisRoutes)` etc. — liga cada grupo de rotas ao seu prefixo.
6. Um middleware 404 para rotas inexistentes.
7. Um middleware de **erro** (4 parâmetros: `err, req, res, next`) que captura falhas inesperadas e responde 500.

> ***Middleware** é uma função que fica "no meio do caminho" entre a requisição e a resposta. A **ordem** em que você os registra importa.*

---

5. Rotas (`routes/`)

Uma rota liga **método HTTP + caminho** a uma função do controller.

```
const router = require("express").Router();
router.get("/", controller.listar);        // GET    /herois
router.get("/:id", controller.buscar);     // GET    /herois/3
router.post("/", controller.criar);        // POST   /herois
router.put("/:id", controller.atualizar);  // PUT    /herois/3
router.delete("/:id", controller.remover); // DELETE /herois/3
```

- O `/` aqui é relativo ao prefixo definido no `app.js` (`/herois`).
- `:id` é um **parâmetro de rota**; o valor chega em `req.params.id` .

Verbos HTTP

Verbo	Significado	CRUD
GET	consultar	**R**ead

POST	criar	**C**reate
PUT	editar	**U**pdate
DELETE	remover	**D**elete

Todas as rotas do projeto

Método	Rota	O que faz
GET	`/herois`	lista heróis
GET	`/herois/:id`	busca um herói
POST	`/herois`	cria herói
PUT	`/herois/:id`	edita herói
DELETE	`/herois/:id`	remove herói
GET	`/herois/:id/itens`	itens de um herói
POST	`/itens`	cria item
PUT	`/itens/:id`	edita item
DELETE	`/itens/:id`	remove item
GET	`/missoes`	lista missões
POST	`/missoes`	cria missão
PUT	`/missoes/:id/status`	alterna o status
DELETE	`/missoes/:id`	remove missão
GET	`/painel`	resumo da guilda

---

6. Controllers (`controllers/`)

Cada função recebe `req` (o pedido) e `res` (a resposta). O padrão se repete em todos os CRUDs:

```
exports.criar = (req, res) => {
  const { nome, classe, nivel = 1 } = req.body;     // 1. ler dados

  if (!nome || !classe)                              // 2. validar
    return res.status(400).json({ erro: "..." });

  const r = db.prepare("INSERT ...").run(nome, classe, nivel); // 3. banco

  res.status(201).json({ id: r.lastInsertRowid, nome, classe, nivel }); // 4. responder
};
```

Códigos de status HTTP usados

Código	Quando
200 OK	deu certo (GET, PUT)
201 Created	registro criado (POST)
204 No Content	removido, sem corpo (DELETE)
400 Bad Request	dados inválidos ou faltando
404 Not Found	herói/item/missão não existe
500 Server Error	erro inesperado

Pontos que merecem atenção

- `return` **após enviar erro.** Sem ele, o código continuaria e tentaria responder duas vezes.
- **Operador** `??` **no PUT.** `req.body.nome ?? heroi.nome` usa o valor novo *se enviado*, senão mantém o antigo. Assim você pode atualizar só o nível, por exemplo.
- **Validar antes de gravar.** Checamos se o herói existe antes de criar um item, para devolver um erro claro (404) em vez de um erro bruto do banco.
- `result.changes` **no DELETE.** Se for 0, nada foi apagado → o id não existia → 404.

Missões: alternar o status

```
const novo = missao.status === "em andamento" ? "concluída" : "em andamento";
```

Um `PUT /missoes/1/status` troca o estado. Chamar de novo volta ao anterior — como o "emprestar/devolver" do enunciado.

Missões: o JOIN

```
SELECT m.*, h.nome AS heroi_nome
FROM missoes m JOIN herois h ON h.id = m.heroi_id
```

`JOIN` combina duas tabelas pela chave estrangeira, trazendo o nome do herói junto com a missão.

Painel: `COUNT(*)`

```
SELECT COUNT(*) AS total FROM missoes WHERE status = 'concluída'
```

Conta linhas. O painel faz quatro contagens e devolve um único objeto JSON.

---

7. O front-end (bônus)

Feito só com HTML, CSS e JavaScript puro.

- `api.js` — função `requisicao()` que usa `fetch` para falar com a API, converte JSON e mostra erros. Todos os outros arquivos a reutilizam.

- **`herois.js`**, **`itens.js`**, **`missoes.js`**, **`painel.js`** — cada um carrega sua lista, desenha o HTML e trata os botões.
- **`e.preventDefault()`** nos formulários — impede o recarregamento da página; o envio é feito por `fetch`.
- **`esc()`** — escapa texto antes de inserir no HTML, evitando **XSS** (um herói chamado `<script>...` não executa código).
- **Ordem dos `<script>`:** `api.js` primeiro, pois os demais dependem dele.

---

8. Como testar (Thunder Client)

Suba o servidor: `cd backend` → `npm start`.

Criar um herói — `POST http://localhost:3000/herois`, Body → JSON:

```
{ "nome": "Aragorn", "classe": "guerreiro", "nivel": 5 }
```

Criar um item — `POST /itens`:

```
{ "nome": "Andúril", "tipo": "arma", "poder": 90, "heroi_id": 1 }
```

Criar uma missão — `POST /missoes`:

```
{ "nome": "Limpar as Minas de Khorval", "heroi_id": 1 }
```

Depois: `PUT /missoes/1/status` e `GET /painel`.

Teste de persistência: pare o servidor (Ctrl+C), suba de novo e faça `GET /herois`. Os dados continuam lá, porque estão no arquivo `guilda.db`.

---

9. Problemas comuns

Sintoma	Causa provável
`req.body` é `undefined`	faltou `app.use(express.json())` ou o header `Content-Type: application/json`
`EADDRINUSE`	já há um servidor usando a porta 3000
`FOREIGN KEY constraint failed`	`heroi_id` não existe, ou faltou ligar o `pragma`
Erro ao instalar `better-sqlite3`	sem binário para sua versão do Node; use uma versão recente do pacote (aqui, `13.0.3`)
Banco vazio após mudar a tabela	`CREATE TABLE IF NOT EXISTS` não altera tabelas já criadas; apague o `.db` em desenvolvimento

---

10. Para evoluir

- Limitar a uma missão ativa por herói (regra de negócio do enunciado).

- Validar `classe` e `tipo` contra listas permitidas.
- Paginação e busca nas listagens.
- Testes automatizados (Jest + Supertest).
- Autenticação para proteger as rotas.

"*Não copie feitiços que você não entende.*" Releia cada arquivo com este guia ao lado.